A.T.L.A.S.

Autonomous Tactical Learning Agentic System

Engineering Documentation

An ontology-driven, autonomously-reasoning operating system for a Solutions Architect
Microsoft 365 Copilot-in-a-browser | Reasoning Brain | 7 concurrent agents | No API keys

Manuel Zelaya
Solutions Architect

Version 1.0 | June 2026
SHI International Corp. | Solutions Engineering

CONFIDENTIAL

TABLE OF CONTENTS

1. ARCHITECTURE OVERVIEW

ATLAS — Autonomous Tactical Learning Agentic System — is an operating system for a Solutions Architect. It fuses two ideas. First, an **ontology-driven operating picture**: every entity the SA works with — customers, meetings, technologies, contacts, and deliverables — is a typed object linked into one navigable model, so the whole account history is a structured graph rather than scattered notes. Second, an **autonomous reasoning agent**: from a single command box it interprets intent, decomposes a goal into steps, dispatches parallel work, and synthesizes the result — acting on the SA's behalf rather than waiting for click-by-click instructions. The **Learning** dimension is cumulative: filed trip reports and artifacts continuously enrich each customer's profile, so the system gets sharper per account over time.

Its inference substrate is **Microsoft 365 Copilot driven through a real browser** (Selenium + persistent Edge/Chrome profiles) — with **no API keys, no tokens, and no admin scopes**. Whatever Copilot can see for the signed-in user, ATLAS can orchestrate. It is a desktop application (pywebview + WebView2) backed by a local HTTP server, a concurrent job engine, a skill registry, and a reasoning Brain.

ATLAS exists because the supported API path to grounded M365 Copilot requires admin consent and licensing that can take many months to obtain. Driving the licensed Copilot web app keeps all customer data inside the user's own Microsoft tenant boundary — no third-party LLM egress.

What ATLAS Does

1. Reads the SA's calendar, meeting transcripts, emails, and files through Copilot **Work** mode
2. Generates structured deliverables via **skills** — trip reports, battlecards, strategic briefings, account 360s, meeting prep, follow-ups, whitespace, topology, knowledge drops
3. Runs a reasoning **Brain** over a free-text command box: classify intent, decompose multi-target goals, and fan out multi-agent **brainwaves**
4. Builds and maintains an **Account Technology Profile** (ATP) per customer from filed trip reports
5. Finds the right SHI engineer/vendor from a **People Resources** directory
6. Runs scheduled instructions on a **Cron** daemon, in the user's timezone
7. Logs **everything** as a searchable artifact, and every brain interaction as a reasoning receipt

Request Lifecycle (Summary)

Layer	Component	Role
1 UI	atlas_web.py · index.html	WebView2 single-page console; polls the API
2 API	atlas/web/{server,api}.py	POST /api/<method> → Api.<method>; returns a job id
3 Brain	atlas/brain/router.py	Classifies intent on Copilot Web vs. the manifest
4 Jobs	atlas/jobs/manager.py	Worker-per-Chrome pool; runs the decision as jobs
5 Engine	atlas/engine/copilot.py	Drives Copilot in the browser (Work/Web)
6 Stores	vault/ + atlas/store/*	Customers, artifacts, ATP, settings, cron

Key Design Decisions

- **Browser-as-brain.** Copilot is the LLM; ATLAS runs threads of it in Work (tenant) or Web (public) mode.
- **Everything is an artifact.** Every deliverable and conversation is logged, readable in-app, searchable, and transferable.

- **Non-destructive.** Deletes are PIN-guarded soft-deletes to vault/recyclebin/; regeneration never clobbers hand-edits (pin-and-merge).
- **Concurrent & resilient.** All real work runs as background jobs; jobs survive navigation and app restarts (vault/jobs.json).
- **Self-aware.** The Brain knows its own capabilities (the manifest) and rules, so it can say what it can do, can't do yet, or can't change.

2. RUN MODEL & LAUNCH

ATLAS launches from `atlas.py` (or `atlas_web.py`). The launcher backfills trip-report artifacts, starts the API and the cron scheduler, then opens the WebView2 window that renders the single-page UI.

```
atlas.py  (launcher)
  |- artifacts.backfill_trip_reports()   # every trip-report .md becomes an artifact
  |- Api()                               # the facade the UI calls
  |- ThreadingHTTPServer 127.0.0.1:<port>  (atlas/web/server.py)
  |- scheduler.start_scheduler()         # Cron Jobs daemon
  +- webview.create_window(...)          # WebView2 (Edge) renders index.html
```

The UI is a **pull model**: the page polls `jobs()`, `days()`, and `artifacts()` roughly every 1.5 seconds and re-renders. Long work never blocks the UI — it is a job on the engine pool.

Prerequisites

- Python 3.10+ with the project virtualenv (.venv) and requirements installed
- Microsoft Edge / Chrome with a signed-in M365 Copilot seed profile (cloned per pool slot)
- A licensed M365 Copilot account — no API keys, tokens, or admin scopes required

3. THE HOME CONSOLE

The Home page is the command console — three columns under a **Working on** job bar (live spinner chips for active jobs, check/warn for finished).

- **Left — Calendar.** Yesterday / Today / Tomorrow toggle. On boot, three gather_day jobs pull each day's meetings (cached to vault/daily/<date>/). A past meeting offers Create Trip Report.
- **Center — the command cluster** (scrollable). Five quick-nav buttons (Customer Vault · People Resources · Cron Jobs · Skills · Artifacts), the animated orb, the Brainwave History button, greeting + live date/time, the 3-row command box (the Brain), the **Continual Suggestions** panel, and — pinned at the bottom — Today's Scheduled Jobs (hidden when none).
- **Right — Recent Artifacts.** Newest-first deliverables (brainwaves excluded — they live in Brainwave History); 'open all' opens the Artifacts browser.

4. SECTIONS & VIEWS

Top bar: Home · Settings · About. Customer Vault / People Resources / Cron Jobs / Skills / Artifacts are the five quick-nav buttons in the Home command cluster.

Section	What it does

Home	The console above.
Artifacts	All deliverables/conversations (brainwaves excluded). Content search + 'Unfiled only' toggle; open & read, File to cus
Customer Vault	Searchable customer grid. A customer shows Trip Reports / ATP / Diagrams / Artifacts, an Overview + Key Contacts
Cron Jobs	Schedule a free-text instruction + time + Daily/Weekly/Monthly; enable/disable/delete.
People Resources	Browse/edit the SHI resource directory (PIN-gated); the find_resource skill.
Brainwave History	Every brain-console interaction as a full reasoning receipt.
Settings	Recipients, signature, auto-email, Timezone, System PIN, Brain Rules, Show Chrome windows.
Skills	Grid of the 12 registered skills; run one directly with its input form.

5. THE BRAIN (ORCHESTRATION)

The command box is a **reasoning router**, not keyword matching. `command(text)` submits a `brain_route` job (Web mode); the Brain classifies intent against its capability manifest and acts.

```
command(text)
  -> JobManager.submit('brain_route', {text}, mode='web')
  -> _brain_route():
       fast-path is_meta?                  -> answer from the manifest (no Copilot call)
       fast-path meeting_trip_reports_day? -> trip_reports_day (regex, for cron reliability)
       else router.classify(ask_web, text) -- Copilot WEB, grounded on the manifest
            -> { action, skill, ctx, items[], collection, range, filter, explanation }
       act on the decision  ->  log a Reasoning receipt to Brainwave History
```

Decision	Behavior
capabilities	Answer 'what can you do / your skills' from the manifest.
run_skill	Spawn the matched skill with extracted ctx (deliverable -> Artifacts).
plan	Decompose a multi-target goal into N parallel skill runs (Section 6).
brainwave	Spawn the multi-agent fan-out (Section 7).
settings	Return the exact pointer; ATLAS never changes its own settings.
cannot_do	Explain why + recommend a skill to add (the 'planned' manifest entries seed this).

Grounding sources. The capability manifest (atlas/brain/capabilities.py) auto-derives skills from the live registry; the Brain Rules (vault/brain/brain_rules.md) are a user-editable constitution injected into prompts (Rule #1: the customer's own saved data comes first).

6. THE PLANNER & TIME REASONING

A skill acts on ONE target. When a request names **multiple** targets, the Brain returns `action=plan` and `_run_plan` fans out one skill job per target in parallel (pool runs 7 at a time, capped at 15/plan), each producing its own artifact, with one Plan receipt in Brainwave History.

- **items[]** — named targets (e.g. 'battlecards for AWC, ProPetro and Sendero' -> 3 jobs).
- **collection** — a set to resolve: yesterday_meetings / today_meetings / tomorrow_meetings / customers / date_range.

Time Reasoning & the Date-Range Coordinator

The classifier is grounded with **today's absolute date**, so it resolves any time expression ('May 21 to June 5', 'last two weeks') into absolute ISO dates and returns `collection=date_range` with a range and an optional `filter=external_customer`. The `skill_over_range` coordinator then:

1. splits the range into ~5-day **windows** (atlas/brain/timespan.py), each gathered in parallel on its own Chrome via gather_window (Copilot Work -> Graph calendar);
2. de-dupes and drops obvious non-meetings (a conservative skip-list), then optionally runs a Copilot-Web pass to keep only external customer meetings;
3. **reduces to the skill's granularity** — a meeting skill (trip_report) fans out per meeting; an account skill (strategic_briefing, battlecard) reduces to distinct customers and fans out per customer;
4. fans out with a sliding-window dependency so only batch_concurrency (default 4) render at once — bounding RAM — capped at 40/job.

So 'trip reports for all my meetings May 21-June 5, external customers only' and 'strategic briefings for any external customer calls last week' both work — any skill over a span, at its natural granularity.

7. BRAINWAVES (MULTI-AGENT FAN-OUT)

For 'is <product> a good fit for <customer>?', the Brain creates a Brainwave History entry immediately (survives interruption), grounds on the customer's own saved data (Rule #1), then fans out **three parallel sub-agents** and synthesizes a grounded verdict.

#	Agent	Mode	Reads
1	Saved data	Web	the customer's actual trip-report content + ATP (chunk-fed)
1	Research	Web	public product research (architecture, competitors, pricing, fit)
1	Live check	Work	live M365 freshness / redundancy not in filed notes
2	Synthesis	Web	merges all three (wave.md) into a grounded verdict + trace

Each sub-agent runs as its own job on a separate Chrome instance; their outputs merge into vault/brain/brainwaves/<id>/wave.md, and the synthesis runs on the facilitator thread. The history entry is then filled in with the verdict and an orchestration trace (which instance ran what).

8. REASONING RECEIPTS

Every brain-console interaction logs a complete **Reasoning receipt** to Brainwave History so the caliber of each interaction can be judged. The receipt always carries the brain's logic; once any spawned work settles, a `finalize_receipt` job (gated on those jobs via after=, so it never blocks a pool worker) enriches it.

- **Brain logic** — the decision (action + skill/targets) and why; that classification ran on Copilot Web.
- **Agent trace** — a table: each agent/skill, Copilot Work/Web mode, the Chrome instance, and output size.
- **Rationale / logic of the answer** — for a single deliverable, a Copilot-Web pass explaining why this is the answer (e.g. why a chosen resource fits).
- **Technology used** — Copilot via the browser pool, and how many Chrome instances.

Childless answers (capabilities / settings / cannot_do) finalize inline and read 'answered from the manifest — 0 Chrome instances,' which itself signals caliber.

9. CONTINUAL SUGGESTIONS

Because ATLAS stays open all day with idle agents and full calendar awareness, the Home screen shows a proactive **next-best-action** panel (`atlas/brain/suggest.py`), pinned above the scheduled-jobs strip. It only **suggests**; nothing runs until you tap a chip, which fires it as a normal, visible job. Two tiers:

- **Tier 1 — instant (free, no Copilot).** Rule-based chips from local signals: the next meeting + its customer (meeting_prep, strategic_briefing), the focus customer, a recent trip-report customer (follow-up), and morning -> daily briefing. Computed fresh on every poll.
- **Tier 2 — strategic (Copilot-Web, throttled).** A `suggest` job reads the customer's saved data and returns up to 3 specific plays (LABEL :: COMMAND), cached to `vault/brain/suggestions.json`.

Refresh, Throttling & the Manual Button

Tier 2 refreshes **event-driven** (a context key of focus-customer or next-meeting changes) plus a **frequency floor** (default 60 min), gated by work-hours and the AI toggle. `Api.suggestions()` serves Tier-1 fresh + Tier-2 cached (cached items show whenever recent, <=6 h — context drift never hides them) and debounce-submits the job when due. `Api.request_suggestions()` backs the **Suggest now** button — forces a run immediately, bypassing the floor / work-hours / AI gate. Every run (manual or auto) leaves a 'Suggestions — …' record in Brainwave History and shows on the Working-on loader.

Settings & Guardrails

Five settings govern it: `continual_suggestions` (master), `suggestions_ai`, `suggestions_frequency_min` (30/60/120/240), `suggestions_focus_customer` (pin one customer, or auto = next meeting), `suggestions_workhours_only`. Guardrails: max 4 chips, dismissible, grounded in the real next-meeting/customer data, **suggest-not-act**, and throttled. Each tile shows a clear label, a faint command preview, and the full command on hover.

10. JOBS & CONCURRENCY

atlas/jobs/manager.py — the JobManager runs one **worker thread per Chrome slot** (POOL_SIZE = 7 -> up to 7 concurrent Copilot instances).

- submit(kind, ctx, title, after=, icon=, mode=) -> a Job{kind, ctx, mode(work|web), status, after[], session_idx, log, result}.
- after= chains jobs; dependents release on any terminal state (_is_settled = done/error/interrupted), so a failed dependency never deadlocks its waiters — this makes the sliding-window throttle safe.
- Persisted to vault/jobs.json (survives restart; in-flight -> 'interrupted'). Idle Chrome auto-closes after 10s.

Job Kinds

gather_day · the 12 skills · atp_generate · brain_route · brainwave · bw_research / bw_cust_saved / bw_cust_live · trip_reports_day · skill_over_range · gather_window · finalize_receipt · suggest (Continual Suggestions) · chat_continue. A plan decision fans out N ordinary skill jobs; skill_over_range does the same, throttled.

11. THE ENGINE (COPILOT-IN-A-BROWSER)

atlas/engine/{pool,copilot}.py. The EnginePool clones chrome_profile_0..6 from a signed-in seed; each is a Session with its own lock.

Session Primitives

Primitive	Purpose
ask(prompt, mode)	One question -> answer turn
ask_chain(prompts)	Several turns in ONE conversation; returns the last good answer
ask_chain_all(prompts)	Every turn's answer (chunked feeds)
ask_at(url, prompt)	Resume a saved conversation by its /chat/conversation/<id> URL

Completion, Guards & Long Context

- **Work | Web** toggle per call (tenant grounding vs. public).
- **Completion** = the ATLAS-RESPONSE-COMPLETE marker, plus accept predicates (substantial >400, brief >80, feed_or_substantial) so Web answers finish even when the marker is dropped.
- **Echo guard** — _best_answer drops any block containing the sentinel hint, so an echoed pasted prompt is never captured as the answer.
- **Long context** is clipboard-pasted (>2000 chars). After an answer, M365 rewrites the URL to /chat/conversation/, captured for resume.
- **Hidden by default** — each Chrome launches off-screen and minimized so the pool never flashes windows; Settings -> 'Show the Copilot Chrome windows' surfaces them for debugging.

12. SKILLS

Skills are self-registering units (atlas/skills/*.py) with typed inputs. The Brain routes to them, or run one directly from the Skills grid. Contract: Skill.run(ctx, ask, log) -> SkillResult.

Skill	Input(s)	Produces
trip_report	meeting, date	Structured trip report from a meeting transcript
meeting_prep	account, meeting	Pre-meeting brief
battlecard	account	Competitive battlecard
strategic_briefing	account	Executive strategic briefing
account_360	account	360-degree account overview
follow_up	account	Follow-up plan / email
whitespace	account	Whitespace / expansion opportunities
environment_topology	account	Topology of the customer's environment
find_resource	need	Best-fit SHI engineer/vendor from People Resources
daily_briefing	(none)	Today's meetings with per-meeting prep
knowledge_drop	(none)	Curated knowledge digest
ask	question	General grounded Q&A (a resumable chat)

13. PEOPLE RESOURCES

A CSV-backed directory of SHI engineers, vendors, assessments, and tool links. atlas/skills/peopleresources/resources.csv is the editable **source of truth** (migrated once from the original .xlsx). The store is atlas/store/people.py.

- **Browse / filter** — category chips + a live filter over name/specialty/region/practice/manager.
- **Edit (PIN-gated)** — add / edit / delete a resource; writes the CSV (the mirror).
- **Import** — drop a new CSV in import/, Import in-app; the prior CSV is archived to archive/ (never destroyed).
- **Brain skill** — find_resource answers 'a TOLA expert who can do PAM or EDR' via deterministic ranking; the Brain adds the rationale in the receipt.

14. ATP SUBSYSTEM (ACCOUNT TECHNOLOGY PROFILE)

atlas/atp/* builds a per-customer technology profile from filed trip reports.

- **Generate** (extract.generate, Web): chunk all trip reports (~20K/turn, one conversation) -> technologies -> topology, a 3-paragraph overview, and key contacts; pin-and-merge keeps hand-edits; saved as a timestamped generation.
- **Relevancy** (recency.py): each tech shows 'appears in N trip reports' + 'last seen', computed from report dates.
- **Topology** (topology_html.py): glowing category columns, status-colored, click-for-specs.
- **Storage**: data_library.py, profile_store.py, generations.py (immutable snapshots).

15. TRIP REPORTS & ARTIFACTS

Every deliverable and conversation is an **artifact** (vault/artifacts.json). Trip reports are first-class: backfilled from existing .md/.mht files at launch, content-searchable, readable in-app, transferable between customers, and emailable via Outlook.

- list_summaries (brainwaves excluded) · list_brainwaves · search · get · backfill_trip_reports · delete (soft, PIN) · uri_id_map
- File to customer files OR transfers an artifact between customers; brainwaves are multifaceted and not filed.

16. CUSTOMER VAULT

Customers live under vault/customers/<slug>/ with trip_reports/, account_technology_profile/ (+ generations), network_diagrams/, artifacts/, and customer.json.

- Create / list / detail / assign_artifact (file or transfer) / delete_customer (PIN -> recyclebin).
- A customer page surfaces Overview + Key Contacts, the ATP editor, Topology, Generate, and Generations picker.

17. CRON SCHEDULER

atlas/jobs/scheduler.py is a daemon (started at launch) that ticks ~every 30s, computes 'now' in the Settings timezone, and fires any due cron by submitting a brain_route job with the instruction — exactly as if typed. So a cron gets the full Brain: planning, date ranges, brainwaves.

- Due = now >= today@HH:MM, the day matches Daily/Weekly/Monthly, and last_fired != today.
- vault/cron.json is the source of truth. Example: '5pm — create trip reports for all of today's external meetings.'

18. STORES & DATA LAYOUT

```
vault/
  customers/<slug>/
    trip_reports/*.md                      # each is an artifact
    account_technology_profile/  (+ generations/<ts>/)
    network_diagrams/  artifacts/  customer.json
  artifacts.json        # deliverables + conversations + brainwaves
  jobs.json             # job history (status bar)
  settings.json         # recipients, signature, tz, system_pin, show_chrome_windows,
                        #   gather_window_size, batch_concurrency
  cron.json             # scheduled instructions
  brain/ brain_rules.md · brainwaves/<id>/wave.md
  recyclebin/           # soft-deleted customers + artifacts (never auto-purged)
  daily/<date>/briefing.json   exports/   inbox/
```

Concurrency-Safe Writes

The pool runs many worker threads that write vault/artifacts.json at once, so the store does an **atomic write** (temp file -> os.replace) under a **module lock** around every load->mutate->save. `_load` is **self-healing**: on a corrupt file it backs up to artifacts.json.corrupt-<UTC>, salvages every intact record (brace-match + de-dupe by id), rewrites a clean file, and never silently returns an empty list for populated-but-broken data.

19. SETTINGS

Setting	Purpose
trip_report_recipients	Comma/semicolon emails the Email button pre-fills
email_signature	Appended under the report body in the draft
auto_email_trip_reports	Auto-open an Outlook draft when a trip report finishes
timezone	IANA timezone the Cron scheduler fires in
system_pin	6-digit PIN required for any delete (blank disables deletion)
show_chrome_windows	Debug: launch the pool's Chrome on-screen (default off = hidden)
gather_window_size	Days per parallel meeting-gather window (default 5)
batch_concurrency	Max deliverables rendering at once in a fan-out (default 4)
continual_suggestions	Master on/off for the Continual Suggestions panel
suggestions_ai	Tier-2 strategic (Copilot) suggestions on/off
suggestions_frequency_min	Min minutes between Tier-2 refreshes (30/60/120/240)
suggestions_focus_customer	Pin advice to one customer; blank = auto (next meeting)

suggestions_workhours_only	Only generate Tier-2 ~7am-7pm
Brain Rules	Editable constitution injected into the Brain's prompts

ATLAS never changes its own settings — the Brain points the user to the Settings page instead.

20. CAPABILITY MANIFEST & BRAIN RULES

The Brain introspects two sources. The **capability manifest** (atlas/brain/capabilities.py) auto-derives skill entries from the live registry plus engine/vault/ATP/settings-pointer/planned entries, so it never drifts. `human_summary()` answers 'what can you do'; `describe()` renders the planner-prompt block.

The **Brain Rules** (vault/brain/brain_rules.md) are a user-editable constitution read on every brainwave and injected into the saved-analysis, freshness, and synthesis prompts. Rule #1: the customer's organized artifacts are the #1 source. Edited in Settings -> Brain Rules.

21. SECURITY & DATA BOUNDARY

- **No API keys, tokens, or admin scopes.** Inference rides the user's licensed Copilot web session.
- **Data stays in tenant.** Customer data does not egress to a third-party LLM API — a key advantage of the browser approach.
- **Destructive actions guarded.** Deletes require a 6-digit System PIN and are soft-deletes to recyclebin/ — never hard deletes.
- **Secrets.** Any credential (e.g. an Entra app secret, if APIs are later enabled) lives only in .env — never in code or the vault.
- **Sign-in recovery.** If a Chrome profile is logged out, the engine surfaces that window on-screen and prompts the user to sign in, then resumes.

22. FILE MAP

```
hackathonsandbox/
  atlas.py / atlas_web.py            launchers
  config.py                          POOL_SIZE, vault dirs, RECYCLEBIN_DIR, CRON_PATH, PEOPLE_*
  atlas/
    web/ server.py · api.py (~40 methods) · index.html
    brain/ capabilities.py · router.py · brainwave.py · rules.py · timespan.py · suggest.py
    jobs/ manager.py · scheduler.py
    engine/ pool.py · copilot.py
    skills/ base.py + 12 skills · peopleresources/ (find_resource + resources.csv)
    store/ vault · customers · artifacts · settings · cron · people
    atp/ extract · generations · recency · topology_html · data_library · profile_store
  vault/                              all runtime data (see Section 17)
  ATLAS_ARCHITECTURE.md / .drawio     master architecture reference + one-page diagram
  SHI_ATLAS_Engineering_Documentation.pdf   (this document)
```

23. CONFIGURATION REFERENCE

config.py

```
POOL_SIZE        = 7        # concurrent Chrome instances (env ATLAS_POOL_SIZE)
VAULT_DIR        = .../vault
CUSTOMERS_DIR    = VAULT_DIR / 'customers'
RECYCLEBIN_DIR   = VAULT_DIR / 'recyclebin'
CRON_PATH        = VAULT_DIR / 'cron.json'
PEOPLE_CSV       = atlas/skills/peopleresources/resources.csv   # source of truth
```

Engine / Accept Predicates

- ATLAS-RESPONSE-COMPLETE marker + accept: substantial (>400 chars), brief (>80), feed_or_substantial.
- Long prompts (>2000 chars) are clipboard-pasted; conversation URL captured for resume.

Brain / Fan-out Knobs

- gather_window_size (5) — days per parallel gather window.
- batch_concurrency (4) — max deliverables rendering at once (sliding-window after[]).
- plan cap 15 targets; date-range fan-out cap 40; span cap ~60 days.

24. TROUBLESHOOTING

Symptom	Cause & Fix
Brainwave History empty	vault/artifacts.json was corrupted; the self-healing loader backs it up (corrupt-<UTC>) and salvages on ne
A Copilot window opens and waits	The M365 session is signed out. Sign in to the surfaced Chrome window; the job resumes.
'No today meetings to report on'	Use an explicit date or range ('May 21 to June 5'); the Brain resolves spans and gathers in parallel.
Chrome windows flash on screen	Settings -> turn OFF 'Show the Copilot Chrome windows' (default hidden/minimized).
A skill ran instead of a rationale	Receipts now always include the rationale + agent trace; open the entry in Brainwave History.
Reports dated wrong	Trip reports are stamped with the meeting's own date, not today's.